

lezione030424

September 22, 2024

#####

Laboratorio di WebScraping

Anno Accademico 2023-2024

Docente: Laura Ricci

Lezione 15

Selenium: gestione di pagine dinamiche

3 Aprile 2024

0.1 Scraping di base: scraping Hacker News con BeautifulSoup

- scraping di **<https://news.ycombinator.com/news>**
- un sito **aggregatore** di news reputate interessante dalla comunità degli hacker
 - computer scientists, data scientists, engineers
- la struttura della pagina è molto semplice

0.2 Scraping di base: scraping Hacker News con BeautifulSoup

0.3 Scraping di base: scraping Hacker News con BeautifulSoup

- una tabella in cui ogni riga (**tag <tr>**), ha **classe athing**
- un elemento della tabella (**tag <td>**), con **classe titleline**, contiene descrizione della notizia e link alla notizia

0.4 Scraping di base: scraping Hacker News con BeautifulSoup

```
[1]: import requests
import re
from bs4 import BeautifulSoup
articles = []
url = 'https://news.ycombinator.com/news'
r = requests.get(url)
html_soup = BeautifulSoup(r.text, 'html.parser')
```

0.5 Scraping di base: scraping Hacker News con BeautifulSoup

```
[2]: import requests
import re
from bs4 import BeautifulSoup

articles = []
url = 'https://news.ycombinator.com/news'
r = requests.get(url)
html_soup = BeautifulSoup(r.text, 'html.parser')

for item in html_soup.find_all('tr', class_='athing'):
    item_title1 = item.find('span', class_='titleline')
    item_title = item_title1.find('a')
    item_title = item_title.get_text(strip=True) if item_title else 'No Link'
    item_link = item_title1.a.get("href")
    articles.append({
        'link' : item_link,
        'title' : item_title,
    })
```

- reperimento titolo e link, un for separato per score e commenti, solo per rendere più chiara la presentazione
- inseriamo informazione reperita in un **Python dictionary**

0.6 Scraping di base: scraping Hacker News con BeautifulSoup

```
[3]: scores=[]
for item in html_soup.find_all('tr', class_='athing'):
    item_title = item.find('span', class_='titleline')
    next_row = item.find_next_sibling('tr')
    item_score = next_row.find('span', class_='score')
    item_score = item_score.get_text(strip=True) if item_score else '0 points'
    # We use regex here to find the correct element
    item_comments = next_row.find('a', string=re.compile('\d+(\&nbsp;
↵|\s)comment(s?)'))
    item_comments = item_comments.get_text(strip=True).replace('\xa0', ' ') \
        if item_comments else '0 comments'
    scores.append({
        'score' : item_score,
        'comments' : item_comments
    })
result = zip(articles[:3], scores[:3])
for res in result:
    print(res)
```

```
({'link': 'https://www.pythonmorsels.com/every-dunder-method/', 'title': 'Every Dunder Method in Python'}, {'score': '34 points', 'comments': '1 comment'})
```

```
({'link': 'https://github.com/intel-analytics/ipex-llm', 'title': 'PyTorch
Library for Running LLM on Intel CPU and GPU'}, {'score': '50 points',
'comments': '6 comments'})
({'link': 'https://redict.io/posts/2024-04-03-redict-7.3.0-released/', 'title':
'Redict 7.3.0, a copyleft fork of Redis, is now available'}, {'score': '117
points', 'comments': '76 comments'})
```

```
<>:8: SyntaxWarning: invalid escape sequence '\d'
<>:8: SyntaxWarning: invalid escape sequence '\d'
C:\Users\ricci\AppData\Local\Temp\ipykernel_15216\2218758285.py:8:
SyntaxWarning: invalid escape sequence '\d'
    item_comments = next_row.find('a',
string=re.compile('\d+(\&nbsp;|\s)comment(s?)'))
```

- uso `find_next_sibling` per passare alla riga successiva
- enormi potenzialità se poi si eseguisse del text mining sul contenuto dei post

0.7 La tecnologia AJAX

- **HackerNoon** è un sito semplice da nalizzare, ma la maggior parte dei siti usa tecnologie di tipo **JavaScript** o **AJAX**
- **AJAX: Asynchronous JavaScript and XML**
 - un insieme di tecniche per lo sviluppo web, non un vero linguaggio di programmazione
 - da la possibilità di effettuare richieste **asincrone** al server, invece di richieste **HTTP** classiche, utilizzando chiamate del tipo **XMLHttpRequest**
 - l'informazione reperita può essere utilizzata per l'aggiornamento **dinamico** di una pagina
- possibilità di **aggiungere** o **aggiornare** il contenuto di una pagina web, senza dover effettuare un **refresh** della pagina
- per migliorare la **user experience**
- uso tipico: siti che incrementano dinamicamente il contenuto mostrato quando si effettua lo **scrolling** della pagina
- qualsiasi social network **Facebook**, **Twitter** utilizzano la tecnica di **infinite scroll pagination**
 - per implementare il **social media feed**
 - quando si effettua uno **scroll down** si carica più contenuto (più post) nella pagina

0.8 La tecnologia AJAX

- pagina di esempio https://scrapingclub.com/exercise/list_infinite_scroll/
- scrollando la pagina si può notare come i capi di abbigliamento vengono caricati dinamicamente

0.9 Web scraping di pagine dinamiche

```
[4]: import requests
from bs4 import BeautifulSoup
url = 'https://scrapingclub.com/exercise/list_infinite_scroll/'
response = requests.get(url)
```

```
html_content = response.text
soup = BeautifulSoup(html_content, 'html.parser')
print(soup.prettify())
```

```
<!DOCTYPE html>
<html class="h-full">
  <head>
    <meta charset="utf-8"/>
    <meta content="width=device-width, initial-scale=1" name="viewport"/>
    <meta content="Learn to scrape infinite scrolling pages" name="description"/>
    <title>
      Scraping Infinite Scrolling Pages (Ajax) | ScrapingClub
    </title>
    <link href="/static/img/icon.611132651e39.png" rel="icon" type="image/png"/>
    <script>
      window.django_app_cfg = {
        dsn: "https://547e0f728838410daf2f1e1a20153fcc@o207778.ingest.sentry.io/4505046779232256",
        release: "2c93d99",
        userPk: "",
      };
    </script>
    <link href="/static/css/app.4a8ac9e71a688f8d087e.76f3a3e8d3f5.css"
rel="stylesheet" type="text/css"/>
    <!-- Google tag (gtag.js) -->
    <script async=""
src="https://www.googletagmanager.com/gtag/js?id=G-BD9ZHFE1XX">
    </script>
    <script>
      window.dataLayer = window.dataLayer || [];

      function gtag() {
        dataLayer.push(arguments);
      }

      gtag('js', new Date());

      gtag('config', 'G-BD9ZHFE1XX');
    </script>
  </head>
  <body class="h-full font-sans antialiased font-normal leading-normal text-gray-700">
    <div class="flex z-50 justify-center py-3 px-4 w-full bg-gray-50 border border-b border-gray-200 lg:py-4 dark:bg-gray-700 dark:border-gray-600 id="banner">
      <div class="items-center md:flex">
        <p class="text-sm font-medium text-gray-900 md:my-0 dark:text-white">
```

```

    <span class="hidden py-0.5 px-2.5 mr-2 text-xs font-semibold text-blue-800
bg-blue-100 rounded md:inline dark:text-blue-800 dark:bg-blue-200">
      New
    </span>
    We have launched Django SaaS Template, which aims to save your time and
money building your product.
    <a class="inline-flex items-center ml-2 text-sm font-medium text-blue-600
md:ml-2 dark:text-blue-500 hover:underline" href="https://saashammer.com/"
target="_blank">
      Check it out
      <svg class="ml-1 w-4 h-4 text-blue-600 dark:text-blue-500"
fill="currentColor" viewBox="0 0 20 20" xmlns="http://www.w3.org/2000/svg">
        <path cliprule="evenodd" d="M10.293 3.293a1 1 0 011.414 0l6 6a1 1 0 010
1.4141-6 6a1 1 0 01-1.414-1.414L14.586 11H3a1 1 0 110-2h11.586l-4.293-4.293a1 1
0 010-1.414z" fillrule="evenodd">
        </path>
      </svg>
    </a>
  </p>
</div>
</div>
<div class="w-full" id="messages">
</div>
<nav class="py-2 px-4 text-sm font-medium bg-white border-b border-gray-200"
data-controller="toggle">
  <div class="container flex flex-wrap justify-between items-center mx-auto
max-w-7xl">
    <div class="flex flex-shrink-0 items-center mr-6">
      <a href="/">
        
      </a>
    </div>
    <div class="block lg:hidden">
      <button class="flex items-center p-3 text-gray-500 lg:hidden focus:text-
gray-700 focus:outline-none" data-action="click->toggle#toggle
touch->toggle#toggle" id="sidebar-open" type="button">
        <svg class="w-5 h-5 fill-current" viewBox="0 0 20 20"
xmlns="http://www.w3.org/2000/svg">
          <title>
            Menu
          </title>
          <path d="M0 3h20v2H0V3zm0 6h20v2H0V9zm0 6h20v2H0v-2z">
          </path>
        </svg>
      </button>
    </div>
    <div class="hidden flex-grow pb-3 w-full lg:flex lg:pb-0 lg:w-auto" data-

```

```

action="native-toggle-nav-bar@window-&toggle#toggle" data-toggle-
target="toggleable">
    <div class="lg:flex-grow">
        <a class="nav-link" href="/">
            Home
        </a>
        <a class="nav-link" href="/#web-scraping-tutorial-using-scrapy">
            Scrapy Tutorial Series
        </a>
        <a class="nav-link" href="/about/">
            About
        </a>
        <a class="nav-link" href="https://leanpub.com/ultimateguidetoscrapy/">
            eBook
        </a>
    </div>
</div>
</div>
</div>
</nav>
<div>
    <div class="container mx-auto max-w-7xl">
        <div class="flex flex-row flex-wrap">
            <div class="px-4 mb-6 w-full sm:w-2/3 md:w-3/4 lg:w-8/12">
                <div class="my-4">
                    <div class="alert alert-success" role="alert">
                        <div class="max-w-full prose">
                            <p>
                                This project was built to help people and I did not earn money from my
work.
                                But you can still support my work
                            </p>
                            <ul>
                                <li>
                                    <a href="//buymeacoff.ee/0JqEZkHre" target="_blank">
                                        Buy me a coffee!
                                    </a>
                                </li>
                                <li>
                                    <a href="https://leanpub.com/ultimateguidetoscrapy/" target="_blank">
                                        Ultimate Guide To Scrapy (eBook)
                                    </a>
                                </li>
                            </ul>
                        </div>
                    </div>
                </div>
            </div>
            <div class="my-4 alert alert-info" role="alert">
                <div class="max-w-full prose">

```

<p>
The key to scrape infinite scrolling pages is to use network panel in your browser to figure out the url of next page.

</p>
<p>
Sometimes you also need to take care of the http headers to make your code work.

</p>
<p>
In this exercise, try to crawl all product info.

</p>
<p>

How to Crawl Infinite Scrolling Pages using Python

</p>
</div>
</div>
<div class="my-4">
<div class="grid grid-cols-1 gap-4 sm:grid-cols-3">
<div class="w-full rounded border post">

<div class="p-4">
<h4>

Short Dress

</h4>
<h5>
\$24.99
</h5>

</div>
</div>
<div class="w-full rounded border post">

<div class="p-4">
<h4>

Patterned Slacks

</h4>

```

        <h5>
            $29.99
        </h5>
    </div>
</div>
<div class="w-full rounded border post">
    <a href="/exercise/list_basic_detail/93926-B/">
        
    </a>
    <div class="p-4">
        <h4>
            <a href="/exercise/list_basic_detail/93926-B/">
                Short Chiffon Dress
            </a>
        </h4>
        <h5>
            $49.99
        </h5>
    </div>
</div>
<div class="w-full rounded border post">
    <a href="/exercise/list_basic_detail/90882-B/">
        
    </a>
    <div class="p-4">
        <h4>
            <a href="/exercise/list_basic_detail/90882-B/">
                Off-the-shoulder Dress
            </a>
        </h4>
        <h5>
            $59.99
        </h5>
    </div>
</div>
<div class="w-full rounded border post">
    <a href="/exercise/list_basic_detail/93756-C/">
        
    </a>
    <div class="p-4">
        <h4>
            <a href="/exercise/list_basic_detail/93756-C/">
                V-neck Top
            </a>
        </h4>

```



```

        <h5>
            $24.99
        </h5>
    </div>
</div>
<div class="w-full rounded border post">
    <a href="/exercise/list_basic_detail/93926-C/">
        
    </a>
    <div class="p-4">
        <h4>
            <a href="/exercise/list_basic_detail/93926-C/">
                Short Chiffon Dress
            </a>
        </h4>
        <h5>
            $49.99
        </h5>
    </div>
</div>
<div class="w-full rounded border post">
    <a href="/exercise/list_basic_detail/93756-B/">
        
    </a>
    <div class="p-4">
        <h4>
            <a href="/exercise/list_basic_detail/93756-B/">
                V-neck Top
            </a>
        </h4>
        <h5>
            $24.99
        </h5>
    </div>
</div>
<div class="w-full rounded border post">
    <a href="/exercise/list_basic_detail/93756-D/">
        
    </a>
    <div class="p-4">
        <h4>
            <a href="/exercise/list_basic_detail/93756-D/">
                V-neck Top
            </a>
        </h4>

```

```

        <h5>
            $24.99
        </h5>
    </div>
</div>
<div class="w-full rounded border post">
    <a href="/exercise/list_basic_detail/96643-A/">
        
    </a>
    <div class="p-4">
        <h4>
            <a href="/exercise/list_basic_detail/96643-A/">
                Short Lace Dress
            </a>
        </h4>
        <h5>
            $59.99
        </h5>
    </div>
</div>
<div class="w-full rounded border post">
    <a href="/exercise/list_basic_detail/94766-A/">
        
    </a>
    <div class="p-4">
        <h4>
            <a href="/exercise/list_basic_detail/94766-A/">
                Fitted Dress
            </a>
        </h4>
        <h5>
            $34.99
        </h5>
    </div>
</div>
</div>
<div class="hidden">
    <nav aria-label="pagination" class="pagination">
        <span class="page active">
            <strong>
                1
            </strong>
        </span>
        <span class="page">
            <a href="/exercise/list_infinite_scroll/?page=2">

```

```

        2
      </a>
    </span>
    <span class="page">
      <a href="/exercise/list_infinite_scroll/?page=3">
        3
      </a>
    </span>
    <span class="page">
      <a href="/exercise/list_infinite_scroll/?page=4">
        4
      </a>
    </span>
    <span class="page">
      <a href="/exercise/list_infinite_scroll/?page=5">
        5
      </a>
    </span>
    <span class="page">
      <a href="/exercise/list_infinite_scroll/?page=6">
        6
      </a>
    </span>
    <span class="page next">
      <a href="/exercise/list_infinite_scroll/?page=2" rel="next">
        Next >
      </a>
    </span>
  </nav>
</div>
<div class="p-16 my-16 bg-gray-50 rounded border border-gray-200">
  <div class="flex justify-center">
    <div class="w-full text-center lg:w-10/12">
      <div class="mb-4 h1">
        Launch Products Faster
        <span class="block text-blue-600">
          with Django
        </span>
      </div>
      <p class="mb-10">
        SaaS Hammer helps you launch products in faster way. It contains all
the foundations you need so you can focus on your product.
      </p>
      <div class="">
        <a class="btn btn-large btn-primary" href="https://saashammer.com/">
          Learn More
        </a>
      </div>
    </div>
  </div>

```

```

    </div>
  </div>
</div>
<div class="px-4 mb-6 w-full sm:w-1/3 md:w-1/4 lg:w-4/12">
  <div class="p-4 my-4 rounded border">
    <div>
      <h3>
        Exercise List
      </h3>
    </div>
    <div class="flex-col space-y-4 max-w-full prose">
      <div class="border-b">
        <p>
          <a class="" href="/exercise/detail_basic/">
            Basic Info Scraping
          </a>
        </p>
        <p>
          Web scraping using XPath or CSS expression
        </p>
      </div>
      <div class="border-b">
        <p>
          <a class="" href="/exercise/detail_json/">
            Analyze JSON
          </a>
        </p>
        <p>
          Load JSON string and extract data
        </p>
      </div>
      <div class="border-b">
        <p>
          <a class="" href="/exercise/list_basic/">
            Recursively Scraping pages
          </a>
        </p>
        <p>
          Not only crawl products but also handle pagination
        </p>
      </div>
      <div class="border-b">
        <p>
          <a class="" href="/exercise/detail_ajax/">
            Mimicking Ajax requests
          </a>
        </p>
      </div>
    </div>
  </div>

```

```

<p>
  Inspect Ajax requests and mimic them
</p>
</div>
<div class="border-b">
  <p>
    <a class="" href="/exercise/detail_header/">
      Inspect HTTP request
    </a>
  </p>
  <p>
    Learn to inspect the fields of HTTP request
  </p>
</div>
<div class="border-b">
  <p>
    <a class="" href="/exercise/list_infinite_scroll/">
      Scraping Infinite Scrolling Pages (Ajax)
    </a>
  </p>
  <p>
    Learn to scrape infinite scrolling pages
  </p>
</div>
<div class="border-b">
  <p>
    <a class="" href="/exercise/detail_cookie/">
      Find gold in cookie
    </a>
  </p>
  <p>
    Make your spider can work with the cookie
  </p>
</div>
<div class="border-b">
  <p>
    <a class="" href="/exercise/basic_login/">
      Login form
    </a>
  </p>
  <p>
    Scrape data behind login form
  </p>
</div>
<div class="border-b">
  <p>
    <a class="" href="/exercise/basic_captcha/">
      Solve Captcha

```

```

        </a>
    </p>
    <p>
        Learn to scrape data behind a captcha
    </p>
</div>
<div class="border-b last:border-b-0">
    <p>
        <a class="" href="/exercise/detail_sign/">
            Decode minified javascript
        </a>
    </p>
    <p>
        Learn how to analyze minimized or compressed javascript
    </p>
</div>
</div>
</div>
</div>
</div>
</div>
<aside class="py-16 bg-gray-50 border-t border-gray-200 lg:py-20" id="footer">
    <div class="px-4 mx-auto max-w-8xl">
        <h2 class="mb-4 text-4xl font-bold tracking-tight text-gray-900 md:font-extrabold md:text-center lg:mb-7 lg:leading-none dark:text-white">
            Sign up for our newsletter
        </h2>
        <p class="mb-6 text-base text-gray-500 md:mb-10 md:text-xl md:text-center lg:px-20 xl:px-56 dark:text-gray-400">
            Subscribe to receive new
            <span class="font-bold">
                web scraping exercises
            </span>
            and
            <span class="font-bold">
                tutorials
            </span>
        </p>
        <div class="mb-4">
            <form action="/newsletter_subscribe/" class="flex max-w-xl md:mx-auto"
            data-action="submit-&gt;subscribe#submit" data-controller="subscribe"
            method="get">
                <div class="w-full">
                    <label class="hidden" for="member_email">
                        Email address
                    </label>
                    <div class="relative h-full">

```

```

        <div class="flex absolute inset-y-0 left-0 items-center pl-3 pointer-
events-none">
            <svg aria-hidden="true" class="w-5 h-5 text-gray-500 dark:text-
gray-400" fill="currentColor" viewBox="0 0 20 20"
xmlns="http://www.w3.org/2000/svg">
                <path d="M2.003 5.884L10 9.882l7.997-3.998A2 2 0 0016 4H4a2 2 0
00-1.997 1.884z">
                </path>
                <path d="M18 8.118l-8 4-8-4V14a2 2 0 002 2h12a2 2 0 002-2V8.118z">
                </path>
            </svg>
        </div>
        <input class="block py-4 px-3 pl-10 w-full text-base text-gray-900 bg-
white rounded-l-xl border border-gray-200 dark:placeholder-gray-400 dark:text-
white dark:bg-gray-700 dark:border-gray-600 focus:border-blue-500 focus:ring-
blue-500 dark:focus:ring-blue-500 dark:focus:border-blue-500" name="email"
placeholder="Enter your email" required="" type="email"/>
    </div>
</div>
<div>
    <input class="py-4 px-4 w-full text-base font-semibold text-center text-
white bg-blue-600 rounded-r-xl border border-blue-600 cursor-pointer hover:bg-
blue-700 focus:border-blue-600 focus:ring-2 focus:ring-blue-600 focus:outline-
none formkit-submit" id="subscribe-submit" type="submit" value="Subscribe"/>
    </div>
</form>
</div>
<p class="hidden text-base text-green-600 md:text-center" id="subscribe-
msg">
    Thanks, please confirm subscription email
</p>
</div>
</aside>
<div class="w-full h-auto text-sm bg-gray-50 border-t border-gray-200
lg:px-6">
    <div class="flex flex-col p-4 px-2">
        <div class="mx-auto text-gray-700">
            Crafted with by
            <a href="https://github.com/michael-yin/">
                MichaelYin
            </a>
        </div>
    </div>
</div>
<script src="/static/js/runtime.733406a7.437561bb6191.js"
type="text/javascript">
</script>
<script src="/static/js/844.c955a86a.8ae7a88d9ece.js" type="text/javascript">

```

```

</script>
<script src="/static/js/app.9eae399d.fbbfa467cd03.js" type="text/javascript">
</script>
<script src="https://unpkg.com/infinite-scroll@4/dist/infinite-
scroll.pkgd.js">
</script>
<script>
  let elem = document.querySelector('.container .grid');
  let infScroll = new InfiniteScroll(elem, {
    // options
    path: 'a[rel="next"]',
    append: '.post',
    history: false,
  });
</script>
</body>
</html>

```

0.10 Web Scraping di pagine dinamiche

- il contenuto dinamico non è presente nella pagina **HTML**
- individuiamo nella pagina la parte relativa allo scroll: si tratta di codice **JavaScript**, che interpreta lo scrolling a emnda chiamata **eAJAX** al server
- come lo acquisisco
 - inutile cercare un link nella pagina per passare alla pagina successiva

0.11 Web Scraping di pagine dinamiche

- è possibile effettuare ancora scraping di questa pagina con **BeutifulSoup?**
- in questo caso, è possibile
 - individuare le chiamate **AJAX** effettuate dal browser per reperire dinamicamente le pagine
 - replicare tali chiamate in Python usando la libreria **Requests**
 - dopo aver caricato tutte le pagine (che si otterrebbero con scroll della pagina) parsare il contenuto

0.12 Pagine dinamiche: https://scrapingclub.com/exercise/list_infinite_scroll/

0.13 Web Scraping di pagine dinamiche: identificare richieste AJAX

- in questo caso occorre però focalizzarsi sul **tab Rete**
- **tab Fetch/XHR**
 - filtra le **XMLHttpRequests**
 - utilizzato per scambiare dati col server web in background, mentre il client sta lavorando in foreground, e aggiornare la parte aggiornata della pagina con i dati ricevuti, senza ricaricare l'intera pagina
 - utilizzato da vari linguaggi di scripting, come JavaScript

- **XML** nel nome è un pò fuorvincente, può essere utilizzato per scambiare dati in altri formati, ad esempio **JSON**
- nel caso del nostro sito refresh della pagina in seguito a scroll
 - viene generata una nuova richiesta per ogni nuovo contenuto caricato mediante scroll
 - ispezionare l'header di ognuna di queste richieste
 - * **URL**, **cookies**, etc.
 - copiare la **URL** per effettuare richieste tramite la libreria **requests**

0.14 Web Scraping di pagine dinamiche: identificare richieste AJAX

0.15 Web Scraping di pagine dinamiche

- dalla analisi delle richieste si nota che
 - ci sono 6 pagine in totale, corrispondenti ai diversi scroll di pagina
- ogni richiesta **XMLHttpRequests** ha il formato
 - `[base_url] + [page_number]`
 - dove `[base_url]` è `https://scrapingclub.com/exercise/list_infinite_scroll/`

0.16 Pagine dinamiche: `https://scrapingclub.com/exercise/list_infinite_scroll/`

- vogliamo individuare i nomi dei prodotti e i prezzi di ogni articolo
- occorre individuare il contenuto all'interno di ogni pagina

0.17 Web Scraping di pagine dinamiche

- notare l'utilizzo della `select` insieme ad un `*CSS selector*`

```
[5]: import requests
from bs4 import BeautifulSoup

base_url = 'https://scrapingclub.com/exercise/list_infinite_scroll/'
page_number = 1 # Start with the base URL
total_pages = 6
while True:
    url = f'{base_url}?page={page_number}'
    response = requests.get(url)
    html_content = response.text
    soup = BeautifulSoup(html_content, 'html.parser')
    products = soup.select('div.p-4 h4 > a')
    prices = soup.select('div.p-4 h5')
    for product, price in zip(products, prices):
        product_name = product.get_text(strip=True)
        product_price = price.get_text(strip=True)
        print(f'Product: {product_name} | Price: {product_price}')
    print('-' * 50)
    page_number += 1
    if page_number > total_pages:
        break
```

Product: Short Dress | Price: \$24.99
Product: Patterned Slacks | Price: \$29.99
Product: Short Chiffon Dress | Price: \$49.99
Product: Off-the-shoulder Dress | Price: \$59.99
Product: V-neck Top | Price: \$24.99
Product: Short Chiffon Dress | Price: \$49.99
Product: V-neck Top | Price: \$24.99
Product: V-neck Top | Price: \$24.99
Product: Short Lace Dress | Price: \$59.99
Product: Fitted Dress | Price: \$34.99

Product: V-neck Jumpsuit | Price: \$69.99
Product: Chiffon Dress | Price: \$54.99
Product: Skinny High Waist Jeans | Price: \$39.99
Product: Super Skinny High Jeans | Price: \$19.99
Product: Oversized Denim Jacket | Price: \$19.99
Product: Short Sweatshirt | Price: \$24.99
Product: Long-sleeved Jersey Top | Price: \$12.99
Product: Skinny High Waist Jeans | Price: \$39.99
Product: Short Sweatshirt | Price: \$24.99
Product: Long-sleeved Jersey Top | Price: \$12.99

Product: Long-sleeved Jersey Top | Price: \$12.99
Product: Jersey Dress | Price: \$19.99
Product: Short Sweatshirt | Price: \$24.99
Product: Crinkled Flounced Blouse | Price: \$24.99
Product: Bib Overall Dress | Price: \$29.99
Product: Loose-knit Sweater | Price: \$17.99
Product: Skinny Regular Jeans | Price: \$29.99
Product: Henley-style Top | Price: \$12.99
Product: Joggers | Price: \$17.99
Product: Skirt with Lacing | Price: \$34.99

Product: Top with Tie | Price: \$17.99
Product: Joggers | Price: \$34.99
Product: Chiffon Dress with Flounce | Price: \$49.99
Product: Skirt with Lacing | Price: \$34.99
Product: V-neck Top | Price: \$9.99
Product: Hooded Jacket | Price: \$29.99
Product: Hooded Top | Price: \$29.99
Product: Lyocell-blend Blouse | Price: \$34.99
Product: Bib Overall Dress | Price: \$29.99
Product: Wide-leg Pants | Price: \$34.99

Product: Ankle-length Slim-fit Pants | Price: \$24.99
Product: Short Sweatshirt | Price: \$9.99
Product: V-neck Blouse | Price: \$17.99
Product: Short Skirt | Price: \$24.99

```

Product: Mesh T-shirt | Price: $17.99
Product: Blouse with Embroidery | Price: $59.99
Product: Wide-cut Cotton Top | Price: $34.99
Product: Pleated Skirt | Price: $54.99
Product: Coat | Price: $69.99
Product: Wide-leg Pants | Price: $49.99
-----
Product: Top | Price: $29.99
Product: Knit Mohair-blend Sweater | Price: $69.99
Product: Dress | Price: $69.99
Product: T-shirt with Embroidery | Price: $29.99
Product: Blouse with Ruffled Collar | Price: $49.99
Product: Pants | Price: $49.99
Product: Jersey Dress | Price: $19.99
Product: T-shirt | Price: $6.99
Product: T-shirt | Price: $6.99
Product: Blazer | Price: $49.99
-----

```

0.18 Web dinamico: JavaScript

- le tre componenti principali del web moderno
 - **HTML**: “fornisce il contenuto”
 - **CSS**: “specifica la presentazione del contenuto”
 - **JavaScript**: “implementa il comportamento, interagendo con l’utente e con il server”
- JavaScript è usato come linguaggio di programmazione client-side dal **98.5% of di tutti i siti web**
 - fonte <https://w3techs.com/technologies/details/cp-javascript>
 - non possiamo ignorarlo se vogliamo effettuare scraping
- diversi livelli d’uso di **JavaScript**
 - rendere le pagine web più interattive e dinamiche
 - gestioni di semplici form
 - pagine interamente dinamiche in cui tutto il contenuto è generato tramite **JavaScript**

0.19 JavaScript in breve

- rilasciato nel 1995, 6 mesi dopo **JAVA**
- inizialmente utilizzato esclusivamente come linguaggio client-side, eseguito nel browser
- arricchito con **Ajax** nel 1999
- introduzione di **Node.js** nel 2009
 - un run-time environment per **JavaScript** open-source, cross-platform
 - possibilità di esecuzione stand-alone o nel server
- attualmente un linguaggio general purpose

0.20 JavaScript e HTML

- il codice **JavaScript** è incorporato nel documento **HTML** attraverso i tag script /script <script>

```
// Il tuo codice JavaScript è inserito qui!!  
</script>
```

oppure si può memorizzare il codice in un file e poi richiamarlo

```
<script src="JSExFiles.js">  
</script>
```

- lo script può essere collocato nella **HEAD section** oppure nella **BODY section**

0.21 JavaScript e HTML

```
<!DOCTYPE html>  
<!-- JSExAlertWrite.html -->  
<html lang="en">  
<head>  
  <meta charset="utf-8">  
  <title>JavaScript Example: Functions alert() and document.write()</title>  
  <script>  
    alert("Hello, world!");  
  </script>  
</head>  
<body>  
  <h1>My first JavaScript says:</h1>  
  <script>  
    document.write("<h2><em>Hello world, again!</em></h2>");  
    document.write("<p>This document was last modified on "  
      + document.lastModified + ".</p>");  
  </script>  
</body>  
</html>
```

- lo script modifica il contenuto della pagina corrente

0.22 JavaScript e HTML

- visualizzazione dell'esempio contenuto nella pagina precedente
- il riferimento, contenuto nel codice, a **document** è un riferimento alla **pagina web corrente**, in cui lo script è integrato
- come risultato della operazione `document.write()` la **BODY section** del documento ora comprende nuovi tag, registrati in coda al documento

```
<h1>My First JavaScript says</h1>  
<h2><em>Hello world, again!</em></h2>  
  <p>This document was last modified on mm/dd/yyyy hh:mm:ss.</p>
```

- il codice **JavaScript** ha modificato il layout dell'**HTML**

0.23 JavaScript: esecuzione nel browser

- un programma **JavaScript** viene inserito in una pagina web
- il browser scarica la pagina del server e lo esegue

- nel browser esiste la macchina virtuale in grado di eseguire il linguaggio
- il programma può utilizzare altre librerie, tra cui **jQuery**
- vengono adottati diversi meccanismi per proteggere il client da attacchi eseguiti dal codice scaricato. Il programma:
 - non può leggere file dalla macchina del client
 - può stabilire connessioni solo con il server a cui si collega, non con altri server
 - non può scrivere sul disco del client
 - non può autonomamente chiudere finestre nel browser che non ha precedentemente aperto

0.24 Quali strumenti per lo scraping di pagine dinamiche?

- in questo corso non ci interessa capire come usare **JavaScript** in ambiente web, ma solo capire come gestire le pagine dinamiche generate da **Javascript** nel processo di scraping!
- **BeautifulSoup + requests** non sono più sufficienti
 - utilizzabili solo quando il codice **JavaScript** è semplice ed è possibile un **reverse engineering** del codice
 - capire quale sono le chiamate al server generate da **JavaScript** e riprodurle in **BeautifulSoup**
 - molto complesso per la maggior parte delle pagine esistenti
- nel caso più generale sono necessari strumenti più sofisticati:
 - **Selenium**

0.25 Selenium: una libreria per lo scraping di pagine dinamiche

- nasce nel 2004 come tool per lo sviluppo di **test automatici** per applicazioni web
 - possibile scrivere script che simulano l'interazione umana con un'applicazione web
 - da la possibilità di
 - * caricare in modo automatico pagine web
 - * simulare azioni come quelle compiute da un utente quando interagisce con la pagina
 - * con lo scopo di **testare** il comportamento del sito web
- successivamente viene utilizzato come ottimo tool per il **web scraping**
 - scraping per pagine dinamiche, ma con funzionalità simili a quelle di BeautifulSoup per l'individuazione di elementi all'interno della pagina
- binding per **Java, Php, Python,...**
- non è una libreria Python standard, installarla con
 - **pip install -U selenium**

0.26 Selenium: il WebDriver

- per interpretare il codice **JavaScript** occorre un vero interprete del linguaggio, che è disponibile solo all'interno di un browser
- si utilizza una tecnologia indicata come **WebDriver**
 - standard di **W3C** dal **2018**
 - un'interfaccia diversa di **Webdriver** a seconda del browser utilizzato
 - gli esempi seguenti si riferiranno a **Google Chrome** e **ChromeDriver**
- il WebDriver è avviato da **Selenium** e successivamente **Selenium** può controllare il browser simulando un comportamento umano

0.27 Selenium: il WebDriver

- tipi diversi di **WebDriver** a seconda del browser utilizzato
- nel caso di **Chrome**
 - lo **zip** scaricato dalla pagine di **Chrome** contiene un eseguibile di nome **chromedriver.exe**
 - * attenzione a scaricare una versione **compatibile** con il vostro browser!
 - Selenium deve essere in grado di accedere a questo eseguibile
 - * inserire direttamente l'eseguibile nella cartella dove si trova lo script o il Notebook Python
 - * oppure indicare esplicitamente il path
- **attivazioni** di tipo diverso a seconda del browser

```
from selenium import webdriver driverChrome = webdriver.Chrome() driverFirefox = web-  
driver.FireFox() driverEdge = webdriver.Edge() driverSafari = webdriver.Safari() driverOpera =  
webdriver.Opera() driverExplorer = webdriver.Ie()
```

0.28 Scraping di una pagina con Selenium: <https://scrapingclub.com/>

0.29 Attivazione del webdriver Chrome

```
[6]: from selenium import webdriver  
import time  
# lancio del driver di Chrome che fa apparire una pagina vuota del browser  
driver=webdriver.Chrome()  
  
# tramite il WebDriver, visita il target site  
url='https://scrapingclub.com/'  
driver.get(url)  
#scraping logic....  
time.sleep(10)  
#rilascia le risorse allocate da Selenium e shut down del browser  
driver.quit()
```

- quando si esegue il codice, appare una nuova finestra vuota tipo quella del browser
 - nella finestra viene caricata la pagina richiesta ...con un certo ritardo....
 - **driver.quit()** per chiudere la finestra e rilasciare le risorse
- la pagina è controllata da **Selenium** invece che da un utente, quindi “pilotata” da **Python**
 - infatti si può notare il messaggio **Chrome is being controller by automated test software**, che avverte che **Selenium** sta controllando questa istanza del browser.
- **Python (Selenium)** può interagire con la pagine con comandi per
 - inserimento di dati nelle form
 - scrolling della pagina
 - click di buttons
 -

0.30 Chrome Headless Mode

- **Headless browser** un browser **senza una interfaccia grafica**, ma con tutte le funzionalità di un vero browser

- utile quando si usa il browser per effettuare **scraping**, oppure per **testare funzionalità** di un sito web
- **Selenium** consente di interagire con un headless browser
 - non vedremo più la finestra di **Chrome**
 - permette di non sprecare risorse nella **GUI**
 - d'altra parte, vedere cosa avviene nel browser può essere utile in fase di debugging, per osservare gli effetti dello script Python direttamente nel browser
 - in fase di produzione, può essere utile eliminare la finestra
- per attivare la modalità **headless**, occorre settare un oggetto **Options** e passarlo al costruttore del WebDriver

0.31 Chrome Headless Mode

```
[7]: from selenium import webdriver
from selenium.webdriver.chrome.options import Options

# enable headless mode in Selenium
options = Options()
options.add_argument('--headless=new')

driver = webdriver.Chrome(
    options=options,
    # other properties...
)

url='https://scrapingclub.com/'
driver.get(url)
#scraping logic....
print("ho aperto un browser headless")
time.sleep(10)
#rilascia le risorse allocate da Selenium e shut down del browser
driver.quit()
```

ho aperto un browser headless

0.32 Selenium: individuare elementi nella pagina HTML

- **Selenium** offre diversi metodi per reperire gli elementi all'interno di una pagina, simili a quelli di **BeautifulSoup**
- **Attenzione!**
 - in **Selenium 4** il nome di molti metodi è cambiato rispetto a **Selenium 3**
 - molti dei metodi che trovate nei testi di riferimento non sono più validi
 - verificare nome e semantica dei metodi in <https://www.selenium.dev/documentation/overview/>
- due funzioni di base per selezionare elementi **HTML** dal **DOM**(Document Object Model)
 - **find_element**: restituisce un elemento specifico
 - **find_elements**: restituisce una lista di elementi che soddisfano una condizione
- entrambe utilizzano gli attributi della classe **By** come selettori dell'elemento

```

find_element(By.ID, "id")
find_element(By.NAME, "name")
find_element(By.XPATH, "xpath")
find_element(By.LINK_TEXT, "link text")
find_element(By.PARTIAL_LINK_TEXT, "partial link text")
find_element(By.TAG_NAME, "tag name")
find_element(By.CLASS_NAME, "class name")
find_element(By.CSS_SELECTOR, "css selector")

```

- tutti i metodi possono sollevare una **NoSuchElementException**

0.33 Selenium: individuare elementi per Text e per TagName

0.34 Selenium: individuare elementi per Text e per TagName

0.35 Selenium: individuare elementi per Class e per CSS Selector

0.36 La tecnologia XPath

- **XPath** indica **XML Path Language**
 - un query language che può essere utilizzato per descrivere un **path** da un nodo **A** ad un nodo **B** in un documento **XML/HTML**
 - fa riferimento alla rappresentazione interna del documento **HTML**, come **DOM**, **Document Object Model**
- simile ai **CSS selectors**, utilizzati per descrivere path che individuano elementi a cui si vogliono applicare **regole di stile**
- vantaggi di **XPath** rispetto ai **CSS selectors**
 - permette di attraversare l'albero in qualsiasi direzione
 - permette trasformazioni dei risultati, prima di restituirli
 - può essere facilmente esteso con ulteriori funzionalità
 - alcune librerie che supportano **CSS selector** convertono i selettori in **XPath**, perchè più veloce e potente

0.37 La tecnologia XPath

```

[8]: from bs4 import BeautifulSoup
html_example = """<div> <a class="button" href="http://pippo.com">MyReference</a> </div>"""
small_soup = BeautifulSoup(html_example, 'html.parser')
print(small_soup.prettify())

```

```

<div>
  <a class="button" href="http://pippo.com">
    MyReference
  </a>
</div>

```

- per riferire l'attributo **href** del nodo **<a>**, che ha una classe **button**, la quale si trova direttamente sotto un nodo **<div>**

0.38 Selenium: individuare elementi per XPath

0.39 Ricercare la card Exercise#1 nella pagina <https://scrapingclub.com/> con XPath

0.40 Ricercare la card Exercise#1 nella pagina <https://scrapingclub.com/> con XPath

- aprire lo **strumento sviluppatori**
- selezionare l'elemento che si vuole analizzare
- selezionare **ispeziona** posizionandosi sull'elemento con il tasto destro
- sulla parte destra del browser appare la pagina dello strumento per sviluppatori e viene evidenziato l'elemento **HTML** corrispondente all'elemento selezionato
- selezionare **copia** posizionandosi sull'elemento **HTML** con il tasto destro
 - copia **selector**
 - copia **XPath**

0.41 Ricercare la card Exercise#1 nella pagina <https://scrapingclub.com/> con XPath

0.42 Ricercare la card Exercise#1 nella pagina <https://scrapingclub.com/> con XPath

```
[11]: from selenium import webdriver
from selenium.webdriver.chrome.options import Options
from selenium.webdriver.common.by import By

# enable headless mode in Selenium
options = Options()
options.add_argument('--headless=new')

driver = webdriver.Chrome(options=options)
url='https://scrapingclub.com/'
driver.get(url)

exercise1_card = driver.find_element(By.CLASS_NAME, 'w-full.rounded.border')
#or
exercise1_card = driver.find_element(By.CSS_SELECTOR, '.w-full.rounded.border')
# or
exercise1_card = driver.find_element(By.XPATH, '/html/body/div[3]/div[2]/div/
↳div[1]')

print(exercise1_card)
```

```
<selenium.webdriver.remote.webelement.WebElement
(session="264e4895c59aa7d05df1aa21bf402f22", element="f.F5C4373A52B170DF9A3188B3
08CE98D8.d.CCAE53E1C9BD17E0C0148301E3B18309.e.6")>
```

0.43 Ricercare tutte le card nella pagina <https://scrapingclub.com/> con XPath

```
[12]: from selenium import webdriver
      from selenium.webdriver.chrome.options import Options
      from selenium.webdriver.common.by import By

      # enable headless mode in Selenium
      options = Options()
      options.add_argument('--headless=new')

      driver = webdriver.Chrome(options=options)
      url='https://scrapingclub.com/'
      driver.get(url)
      exercise_cards = driver.find_elements(By.CLASS_NAME, 'w-full.rounded.border')
      #or
      exercise_cards = driver.find_elements(By.CSS_SELECTOR, '.w-full.rounded.border')
      exercise_cards
```

```
[12]: [<selenium.webdriver.remote.webelement.WebElement
      (session="357fab41d90a961732c3b7a1c52e6671", element="f.450DBD42B87DA9AF34DF90B6
      8C1CC743.d.BCE1D3DA2A191C3D846045BDE598DED1.e.6")>,
      <selenium.webdriver.remote.webelement.WebElement
      (session="357fab41d90a961732c3b7a1c52e6671", element="f.450DBD42B87DA9AF34DF90B6
      8C1CC743.d.BCE1D3DA2A191C3D846045BDE598DED1.e.7")>,
      <selenium.webdriver.remote.webelement.WebElement
      (session="357fab41d90a961732c3b7a1c52e6671", element="f.450DBD42B87DA9AF34DF90B6
      8C1CC743.d.BCE1D3DA2A191C3D846045BDE598DED1.e.8")>,
      <selenium.webdriver.remote.webelement.WebElement
      (session="357fab41d90a961732c3b7a1c52e6671", element="f.450DBD42B87DA9AF34DF90B6
      8C1CC743.d.BCE1D3DA2A191C3D846045BDE598DED1.e.9")>,
      <selenium.webdriver.remote.webelement.WebElement
      (session="357fab41d90a961732c3b7a1c52e6671", element="f.450DBD42B87DA9AF34DF90B6
      8C1CC743.d.BCE1D3DA2A191C3D846045BDE598DED1.e.10")>,
      <selenium.webdriver.remote.webelement.WebElement
      (session="357fab41d90a961732c3b7a1c52e6671", element="f.450DBD42B87DA9AF34DF90B6
      8C1CC743.d.BCE1D3DA2A191C3D846045BDE598DED1.e.11")>,
      <selenium.webdriver.remote.webelement.WebElement
      (session="357fab41d90a961732c3b7a1c52e6671", element="f.450DBD42B87DA9AF34DF90B6
      8C1CC743.d.BCE1D3DA2A191C3D846045BDE598DED1.e.12")>,
      <selenium.webdriver.remote.webelement.WebElement
      (session="357fab41d90a961732c3b7a1c52e6671", element="f.450DBD42B87DA9AF34DF90B6
      8C1CC743.d.BCE1D3DA2A191C3D846045BDE598DED1.e.13")>,
      <selenium.webdriver.remote.webelement.WebElement
      (session="357fab41d90a961732c3b7a1c52e6671", element="f.450DBD42B87DA9AF34DF90B6
      8C1CC743.d.BCE1D3DA2A191C3D846045BDE598DED1.e.14")>,
      <selenium.webdriver.remote.webelement.WebElement
      (session="357fab41d90a961732c3b7a1c52e6671", element="f.450DBD42B87DA9AF34DF90B6
```

8C1CC743.d.BCE1D3DA2A191C3D846045BDE598DED1.e.15")>]

0.44 Reperire i prodotti nella prima pagina

- verificare come il programma illustrato mostrato sotto restituisca solo i vestiti presenti nella prima pagina
- per veder i prodotti c'è bisogno di una procedura che **automatizzi** il processo di scrolling

```
[13]: from selenium import webdriver
      from selenium.webdriver.common.by import By
      import time
      driver = webdriver.Chrome()
      driver.get('https://scrapingclub.com/exercise/list_infinite_scroll/')

      captured_dress = driver.find_elements(By.CLASS_NAME, 'p-4')
      for dress in captured_dress:
          try:
              description=dress.find_element(By.TAG_NAME, 'h4').text
              print(description)
              price=dress.find_element(By.TAG_NAME, 'h5').text
              print(price)
          except:
              pass
```

```
Short Dress
$24.99
Patterned Slacks
$29.99
Short Chiffon Dress
$49.99
Off-the-shoulder Dress
$59.99
V-neck Top
$24.99
Short Chiffon Dress
$49.99
V-neck Top
$24.99
V-neck Top
$24.99
Short Lace Dress
$59.99
Fitted Dress
$34.99
```

0.45 Reperire tutti i prodotti: scrolling, prima soluzione

```
[14]: from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver import Keys
from selenium.webdriver.common.action_chains import ActionChains
import time

driver = webdriver.Chrome()
driver.get('https://scrapingclub.com/exercise/list_infinite_scroll/')

def scroll_to_bottom():
    # driver.find_element(By.TAG_NAME, 'body')
    # Define the number of times to scroll (adjust as needed)
    scroll_iterations = 5
    for _ in range(scroll_iterations):
        # Scroll down inviando la **END key** al browser
        ActionChains(driver).send_keys(Keys.END).perform()
        # Aspetta un poco per garantire che il contenuto sia stato caricato
        time.sleep(5)
```

0.46 Reperire tutti i prodotti: scrolling, prima soluzione

```
[15]: # Call the function to scroll to the bottom
scroll_to_bottom()

captured_dress = driver.find_elements(By.CLASS_NAME, 'p-4')

for dress in captured_dress:
    try:
        description=dress.find_element(By.TAG_NAME, 'h4').text
        print(description)
        price=dress.find_element(By.TAG_NAME, 'h5').text
        print(price)
    except:
        pass
```

```
Short Dress
$24.99
Patterned Slacks
$29.99
Short Chiffon Dress
$49.99
Off-the-shoulder Dress
$59.99
V-neck Top
$24.99
```

Short Chiffon Dress
\$49.99
V-neck Top
\$24.99
V-neck Top
\$24.99
Short Lace Dress
\$59.99
Fitted Dress
\$34.99
V-neck Jumpsuit
\$69.99
Chiffon Dress
\$54.99
Skinny High Waist Jeans
\$39.99
Super Skinny High Jeans
\$19.99
Oversized Denim Jacket
\$19.99
Short Sweatshirt
\$24.99
Long-sleeved Jersey Top
\$12.99
Skinny High Waist Jeans
\$39.99
Short Sweatshirt
\$24.99
Long-sleeved Jersey Top
\$12.99
Long-sleeved Jersey Top
\$12.99
Jersey Dress
\$19.99
Short Sweatshirt
\$24.99
Crinkled Flounced Blouse
\$24.99
Bib Overall Dress
\$29.99
Loose-knit Sweater
\$17.99
Skinny Regular Jeans
\$29.99
Henley-style Top
\$12.99
Joggers
\$17.99

Skirt with Lacing
\$34.99
Top with Tie
\$17.99
Joggers
\$34.99
Chiffon Dress with Flounce
\$49.99
Skirt with Lacing
\$34.99
V-neck Top
\$9.99
Hooded Jacket
\$29.99
Hooded Top
\$29.99
Lyocell-blend Blouse
\$34.99
Bib Overall Dress
\$29.99
Wide-leg Pants
\$34.99
Ankle-length Slim-fit Pants
\$24.99
Short Sweatshirt
\$9.99
V-neck Blouse
\$17.99
Short Skirt
\$24.99
Mesh T-shirt
\$17.99
Blouse with Embroidery
\$59.99
Wide-cut Cotton Top
\$34.99
Pleated Skirt
\$54.99
Coat
\$69.99
Wide-leg Pants
\$49.99
Top
\$29.99
Knit Mohair-blend Sweater
\$69.99
Dress
\$69.99

T-shirt with Embroidery
\$29.99
Blouse with Ruffled Collar
\$49.99
Pants
\$49.99
Jersey Dress
\$19.99
T-shirt
\$6.99
T-shirt
\$6.99
Blazer
\$49.99

0.47 Reperire tutti i prodotti: scrolling, seconda soluzione

```
[16]: from selenium import webdriver
      from selenium.webdriver.common.by import By
      from selenium.webdriver import Keys
      from selenium.webdriver.common.action_chains import ActionChains
      import time

      driver = webdriver.Chrome()
      driver.get('https://scrapingclub.com/exercise/list_infinite_scroll/')

      def scroll_to_bottom():
          cards = driver.find_elements(By.CLASS_NAME, 'p-4')
          old_cards = len(cards)
          while True:
              # Get the body element to perform actions on
              driver.find_element(By.TAG_NAME, 'body')
              # Define the number of times to scroll (adjust as needed)
              ActionChains(driver).send_keys(Keys.END).perform()
              time.sleep(5)
              cards = driver.find_elements(By.CLASS_NAME, 'w-full')
              if (old_cards == len(cards)):
                  # break the cycle since the scroll
                  # is over
                  break
              # keep track of the number of cards
              # currently discovered
              old_cards = len(cards)
```

0.48 Reperire tutti i prodotti: scrolling, seconda soluzione

```
[17]: # Call the function to scroll to the bottom

scroll_to_bottom()

captured_dress = driver.find_elements(By.CLASS_NAME, 'p-4')
for dress in captured_dress:
    try:
        description=dress.find_element(By.TAG_NAME, 'h4').text
        print(description)
        price=dress.find_element(By.TAG_NAME, 'h5').text
        print(price)
    except:
        pass
```

Short Dress
\$24.99
Patterned Slacks
\$29.99
Short Chiffon Dress
\$49.99
Off-the-shoulder Dress
\$59.99
V-neck Top
\$24.99
Short Chiffon Dress
\$49.99
V-neck Top
\$24.99
V-neck Top
\$24.99
Short Lace Dress
\$59.99
Fitted Dress
\$34.99
V-neck Jumpsuit
\$69.99
Chiffon Dress
\$54.99
Skinny High Waist Jeans
\$39.99
Super Skinny High Jeans
\$19.99
Oversized Denim Jacket
\$19.99
Short Sweatshirt
\$24.99

Long-sleeved Jersey Top
\$12.99
Skinny High Waist Jeans
\$39.99
Short Sweatshirt
\$24.99
Long-sleeved Jersey Top
\$12.99
Long-sleeved Jersey Top
\$12.99
Jersey Dress
\$19.99
Short Sweatshirt
\$24.99
Crinkled Flounced Blouse
\$24.99
Bib Overall Dress
\$29.99
Loose-knit Sweater
\$17.99
Skinny Regular Jeans
\$29.99
Henley-style Top
\$12.99
Joggers
\$17.99
Skirt with Lacing
\$34.99
Top with Tie
\$17.99
Joggers
\$34.99
Chiffon Dress with Flounce
\$49.99
Skirt with Lacing
\$34.99
V-neck Top
\$9.99
Hooded Jacket
\$29.99
Hooded Top
\$29.99
Lyocell-blend Blouse
\$34.99
Bib Overall Dress
\$29.99
Wide-leg Pants
\$34.99

Ankle-length Slim-fit Pants
\$24.99
Short Sweatshirt
\$9.99
V-neck Blouse
\$17.99
Short Skirt
\$24.99
Mesh T-shirt
\$17.99
Blouse with Embroidery
\$59.99
Wide-cut Cotton Top
\$34.99
Pleated Skirt
\$54.99
Coat
\$69.99
Wide-leg Pants
\$49.99
Top
\$29.99
Knit Mohair-blend Sweater
\$69.99
Dress
\$69.99
T-shirt with Embroidery
\$29.99
Blouse with Ruffled Collar
\$49.99
Pants
\$49.99
Jersey Dress
\$19.99
T-shirt
\$6.99
T-shirt
\$6.99
Blazer
\$49.99

0.49 Selenium: interagire con il browser

0.50 Selenium: interagire con il browser

<

>

0.51 Selenium: interagire con il browser

- simulare la selezione di un **radio button** dell'utente
- viene ricercato nel codice **HTML** il **button** corrispondente al giorno della settimana che si vuole selezionare
 - si usa una ricerca per **XPATH**
 - **XPATH** individuato ispezionando nel browser l'elemento selezionato
 - se ne simula il **click**

```
[18]: from selenium import webdriver
from selenium.webdriver.common.keys import Keys
from selenium.webdriver.common.by import By
import time

# Create chrome driver.
driver = webdriver.Chrome()

driver.implicitly_wait(1000)

# Open the webpage.
driver.get("https://openwritings.net/sites/default/files/selenium-test-pages/
↳radio_checkbox.html")

# Radio Button: Check Monday using XPATH locator.
element = driver.find_element(By.XPATH, "//*[@value='Mon']")
element.click()
print("primo click")
```

primo click

0.52 Selenium: interagire con il browser

- simulare ora la selezione di due **Check boxes**
- si usa una ricerca per **CSS_SELECTOR**
- se ne simula la selezione

```
[19]: # Checkbox: Uncheck Apple using CSS selector.
element2=driver.find_element(By.CSS_SELECTOR, "input[name='apple']")
element2.click()
print("secondo click")

# Checkbox: Check Orange using CSS selector.
element3=driver.find_element(By.CSS_SELECTOR, "input[name='orange']")
element3.click()
print("terzo click")

# Pause for 10 seconds so that you can see the results.
time.sleep(10)
```

```
# Close.  
driver.close()
```

secondo click

terzo click

0.53 Selenium: interagire automaticamente con il browser

- consideriamo una form più complessa, come la seguente

0.54 Selenium: interagire automaticamente con il browser

0.55 Selenium: interagire automaticamente con il browser

```
[20]: from selenium import webdriver  
from selenium.webdriver.support.select import Select  
from selenium.webdriver.common.keys import Keys  
from selenium.webdriver.common.by import By  
  
url = 'http://www.webscrapingfordatascience.com/postform2/'  
driver = webdriver.Chrome()  
driver.implicitly_wait(10)  
driver.get(url)  
driver.find_element(By.NAME, 'name').send_keys('Laura')  
print("inserito il nome")
```

inserito il nome

0.56 Selenium: interagire automaticamente con il browser

```
[21]: driver.find_element(By.CSS_SELECTOR, 'input[name="gender"][value="F"]').click()  
print("cliccato il gender")  
driver.find_element(By.NAME, 'pizza').click()  
print("cliccato meal")  
driver.find_element(By.NAME, 'salad').click()  
print("cliccato 2 meal")  
Select(driver.find_element(By.NAME, 'haircolor')).select_by_value('brown')  
print("selezionati i capelli")  
driver.find_element(By.NAME, 'comments').send_keys(  
    ['First line', Keys.ENTER, 'Second line'])
```

cliccato il gender

cliccato meal

cliccato 2 meal

selezionati i capelli

0.57 Selenium: interagire automaticamente con il browser

```
[22]: input('Press ENTER to submit the form')
driver.find_element(By.TAG_NAME, 'form').submit()
# Or: driver.find_element_by_css_selector('input[type="submit"]').click()
input('Press ENTER to close the automated browser')
driver.quit()
```

Press ENTER to submit the form

Press ENTER to close the automated browser

0.58 Selenium: sincronizzazione su contenuti dinamici

- il sito: URL: https://the-internet.herokuapp.com/dynamic_loading/1
- comportamento
 - il sito visualizza un botton
 - l'utente clicca sul bottone
 - il sito mostra una animazione che indica che si sta caricando del contenuto
 - infine si stampa il messaggio "Hello World!".

0.59 Selenium: sincronizzazione su contenuti dinamici

0.60 Selenium: il problema della sincronizzazione

```
[23]: # Required imports
from selenium import webdriver
from selenium.webdriver.common.by import By

#Initialize webdriver
driver = webdriver.Chrome()

#Open website
driver.get('https://the-internet.herokuapp.com/dynamic_loading/1')
elem = driver.find_element(By.XPATH, '//*[@id="start"]/button')
elem.click()
elem2=driver.find_element(By.ID, "finish")
print(elem2.text)
```

- perchè non stampa "Hello world?"

0.61 Il problema della sincronizzazione

- esiste un problema di **temporizzazione**
 - il browser impiega un pò di tempo per effettuare eventuali chiamate **AJAX** e per eseguire il codice **JavaScript**
 - l'esecuzione è **asincrona** rispetto allo scraper Python
 - l'esecuzione del metodo che implementa il parsing inizia prima che tale esecuzione sia terminata

- il browser accede ad una pagina che non è ancora consistente
- questo rende difficile la sincronizzazione tra il **WebDriver** e il **browser**
- se un elemento non è ancora stato caricato
 - la funzione che lo localizza può sollevare una **ElementNotVisibleException** o **NoSuchElementException**
- è necessario utilizzare meccanismi che consentano di implementare una **corretta sincronizzazione**

0.62 Selenium: sincronizzare Python con il browser

```
[24]: # Required imports
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC

#Initialize webdriver
driver = webdriver.Chrome()
#Open website
driver.get('https://the-internet.herokuapp.com/dynamic_loading/1')
elem = WebDriverWait(driver, 20).until(EC.element_to_be_clickable((By.XPATH, '//
↳*[id="start"]/button'))) #Find element and wait for it to be clickable
elem.click() #click on element
elem2 = WebDriverWait(driver, 20).until(EC.visibility_of_element_located((By.
↳ID, "finish"))) #Find element by text
print(elem2.text) #Display text
```

Hello World!

0.63 Selenium: parsing con sincronizzazione esplicita

```
from selenium.webdriver.support.wait import WebDriverWait
elem = WebDriverWait(driver, tempo).until(condizione)
```

- il thread che esegue lo scraper viene interrotto finché non si verifica una condizione
- l'esecuzione riprende se:
 - si verifica la condizione specificata
 - si raggiunge un tempo massimo per l'attesa
- implementa una sincronizzare **sullo stato** tra il WebDriver e il browser
- vedremo meglio le primitive di sincronizzazione in una lezione successiva

[]: